TRABALHO PRÁTICO

Ana Clara Vasconcelos dos Santos e Isabela Ramos dos Santos

2021080573 e 2023034757

aninhavds1@ufmg.br e isasantosufmg@gmail.com

1. INTRODUÇÃO

Este trabalho apresenta um sistema de busca geoespacial de entregadores para Belo Horizonte, construído sobre duas estruturas implementadas do zero: uma Patricia Trie e um codec Geohash. Uma segunda instância da Trie (classe TrieTextualBares) faz a busca textual de estabelecimentos por nome, rua ou bairro.

O problema central é localizar, de forma eficiente, os entregadores dentro de um raio definido pelo usuário (padrão de 2 km) a partir de um bar ou restaurante. O Geohash discretiza o espaço contínuo em células identificadas por strings, e a Patricia Trie indexa essas strings, explorando a propriedade de prefixo compartilhado para reduzir a busca de N entregadores para apenas os das células relevantes.

A solução está em Python, com interface Dash/Plotly. As estruturas centrais (Trie, Geohash, Haversine) são próprias. As bibliotecas osmnx, networkx, geopandas, pandas, numpy e shapely servem apenas de apoio para carregar dados geográficos oficiais e calcular rotas.

2. MODELAGEM

2.1 Descrição do Problema

Dado um conjunto de N entregadores sobre a malha viária de BH, cada um com uma posição, como responder eficientemente: quais entregadores estão a no máximo R km de um ponto P?

A estratégia usa duas camadas: (1) discretização por Geohash, que preserva localidade em strings; (2) uma Patricia Trie sobre os geohashes de precisão 8 dos entregadores, que recupera todos os entregadores de uma célula (ou de um conjunto de células adjacentes) por busca de prefixo.

Há ainda o efeito de borda: um entregador pode estar dentro do raio e em outra célula. O sistema consulta também as 8 células vizinhas da célula do ponto de consulta e filtra pela distância real (Haversine).Essa estratégia só cobre todo o disco de busca quando o raio não excede o menor lado da célula usada; a seção 2.5 quantifica a limitação.

2.2 Representação dos Dados

Cada entregador é um dicionário Python:

Tipo	Descrição	Campo
int	Identificador sequencial	id
float	Coordenadas WGS-84 do nó viário (y e x no grafo OSM)	lat, lon
float	Aliases de lat e lon (compatibilidade)	latitude, longitude

Tipo	Descrição	Campo
str	Geohash de precisão 8	geohash
int	Bar em torno do qual o entregador foi gerado	id_bar_central

Cada bar é carregado por `carregar_bares()` (backend/bares.py), que lê `data/output/bares_com_geohash.xlsx` e descarta linhas sem latitude ou longitude. O dicionário de cada bar contém: id (coluna _id ou posição), nome (prioridade: nome fantasia, depois razão social, depois logradouro), rotulo_dropdown, nome_fantasia, razao_social, bairro, logradouro (tipo, nome e número), endereco_completo, lat, lon, latitude, longitude e geohash. Na interface, o valor selecionado no dropdown é o índice do bar na lista BARES, não o campo id.

Todos os entregadores têm a posição corrigida para o nó mais próximo da malha viária, de modo que nenhum fica em local inacessível a veículos.

2.3 Geohash

O Geohash converte um par (latitude, longitude) em uma string no alfabeto base32 (*0123456789bcdefghjkmnpqrstuvwxyz*, sem a, i, l, o). A implementação própria está em backend/geohash.py. A função `encode()` divide alternadamente os intervalos de longitude (bits de índice par) e latitude (bits ímpares): para P caracteres geram-se 5P bits, e cada grupo de 5 bits vira um caractere. A precisão aceita é de 1 a 8, com validação das entradas.

A figura a seguir ilustra o processo para os 5 primeiros bits de longitude de um ponto hipotético:

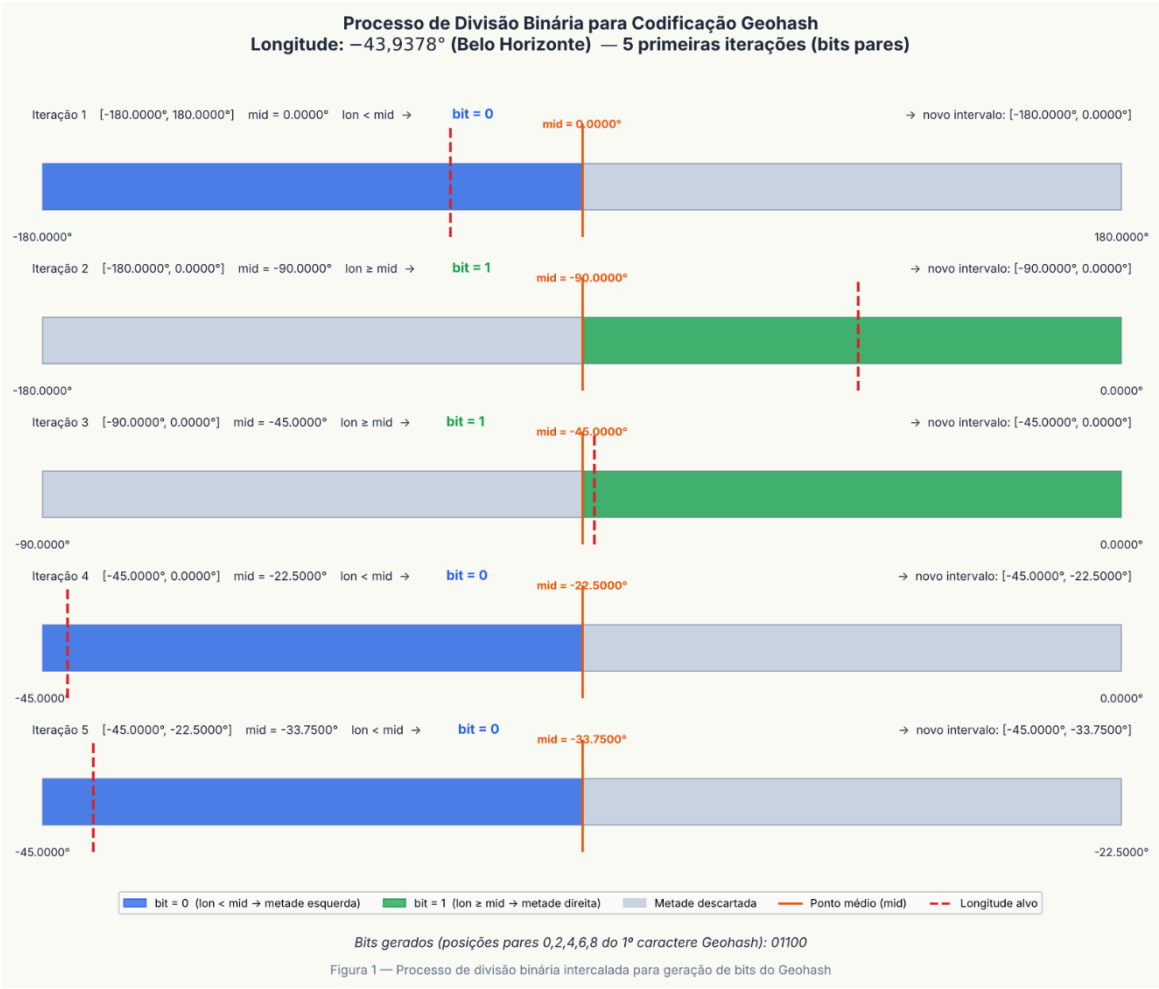


Figura 1 — Processo de divisão binária intercalada para geração de bits do Geohash

Tamanho das células em BH (medido com `decode_geohash`):

Precisão	Altura (lat)	Largura (lon)	Uso no sistema
5	~4,9 km	~4,6 km	Busca com raio maior que 1,2 km
6	~0,61 km	~1,15 km	Busca com raio de até 1,2 km
8	~19 m	~36 m	Armazenamento na Trie

Prefixos de BH. Amostrando pontos da caixa envolvente do município, todos caem em 7h (P=2) e 7h2 (P=3); em P=4 há 6 células (7h2t a 7h2z) e em P=5 cerca de 35. A versão anterior citava também 7h3, que não ocorre.

Propriedade de prefixo. O geohash de menor precisão é prefixo do de maior precisão para o mesmo ponto. Por exemplo, encode(-19.917, -43.934, 6) = 7h2y8q, e seu prefixo 7h2y8 é o geohash de precisão 5. Assim, buscar na Trie por um geohash de precisão 5 ou 6 recupera todos os entregadores (armazenados com precisão 8) da célula.

decode_geohash reconstrói os intervalos de latitude e longitude, devolvendo centro e erro (metade do tamanho da célula). **haversine** (R = 6371 km) também está em geohash.py. A figura de divisão binária da versão anterior (Figura 1 do PDF original) permanece válida.

2.4 Patricia Trie

Implementada em backend/trie.py. Cada nó guarda um rótulo (prefixo comprimido), um dicionário filhos (primeiro caractere do rótulo do filho → nó) e uma lista valores, com **slots** para economizar memória.

```
class Node:
    __slots__ = (rotulo, filhos, valores)
```

(O código real declara os três nomes como strings no **slots**.)

Inserção `(addNode)`**.** Compara a chave restante com o rótulo do filho usando longest_prefix(). Três casos: (1) não há filho com aquela letra, então cria-se uma folha; (2) o prefixo comum esgota o rótulo, então desce-se ao filho; (3) o prefixo comum é parcial, então o filho é dividido em um nó intermediário com dois ramos. Se a chave termina exatamente no nó, o dado entra na lista valores.

Busca `(searchNode)`**.** Percorre a árvore com o prefixo; se o prefixo termina no meio do rótulo de um filho, aquele filho é a raiz da subárvore coletada; se diverge, retorna lista vazia. coletar() faz DFS iterativa com pilha explícita, evitando estouro de recursão.

Chaves iguais não criam nós novos: entregadores que a correção viária levou ao mesmo nó do grafo têm o mesmo geohash e ficam na mesma lista valores. Logo, a Trie não tem necessariamente N folhas; tem no máximo N nós terminais mais os intermediários.

2.5 Tratamento das Bordas

Algoritmo atual (vizinhos em `backend/geohash.py`**).** Decodifica o geohash para obter o centro e o erro (lat_err, lon_err); desloca o centro em 8 direções; recodifica cada ponto com a mesma precisão. As coordenadas deslocadas são limitadas a [-90, 90] e [-180, 180] antes de encode(), evitando exceção perto dos limites do globo.

```
def vizinhos(geohash: str) -> list[str]:
    precisao = len(geohash)
    lat_c, lon_c, lat_err, lon_err = decode_geohash(geohash)
    passo_lat = lat_err * 2.1
    passo_lon = lon_err * 2.1
    # ... 8 direções (N, S, L, O e diagonais)
    celulas = {geohash}
    for dlat, dlon in direcoes:
        nova_lat = max(-90.0, min(90.0, lat_c + dlat))
        nova_lon = max(-180.0, min(180.0, lon_c + dlon))
        celulas.add(encode(nova_lat, nova_lon, precisao))
    return list(celulas)
```

O fator 2,1: lat_err e lon_err são a metade do tamanho da célula. Então 2,1 × erro = 1,05 célula: o ponto deslocado cai próximo do centro da célula vizinha, longe de suas fronteiras (a versão anterior interpretava o fator como 2,1 células e a faixa 2 a 3 como margem, o que não procede). Os 5% extras só protegem contra erros de ponto flutuante.

Limite de cobertura: O bloco 3 × 3 cobre, a partir de qualquer ponto da célula central, pelo menos um lado de célula em cada direção. Se o raio for maior que o menor lado da célula, parte do disco fica fora do bloco . Portanto, com a regra atual (P=6 até 1,2 km; P=5 acima disso), a consulta é exata para raios até ~0,6 km (P=6) e para raios entre 1,2 e ~4,6 km (P=5). Para raios intermediários e grandes (a interface aceita até 50 km) ela pode omitir entregadores. A correção recomendada está na seção 7.

3. IMPLEMENTAÇÃO

3.1 Estruturas de Dados

- **Patricia Trie geoespacial** (`trie.py`: Node e Trie): descrita na seção 2.4.
- **Patricia Trie textual** (`trie.py`: TrieTextualBares, subclasse de Trie)Para cada bar, normaliza (minúsculas, sem acentos e pontuação) o texto formado por nome de apresentação, razão social, logradouro e bairro, e indexa cada palavra de 2 ou mais letras. Além disso, indexa o nome de apresentação sem espaços (3 ou mais letras), o que permite achar por nome colado. A busca (buscar) usa apenas a última palavra digitada como prefixo, remove índices repetidos e limita o número de opções (60 na interface). Com menos de 2 caracteres, devolve as 40 primeiras opções (obter_opcoes_iniciais). O rótulo exibido é nome — rua, bairro (formatar_rotulo); o campo rotulo_dropdown de bares.py não é usado pelo dropdown.
- **Geohash próprio** (`geohash.py`): encode, decode_geohash, vizinhos e haversine. A versão anterior citava uma suíte de 25 testes em tests/test_geohash.py; esses arquivos não foram enviados nesta revisão e a afirmação não pôde ser reverificada.
- **Carregador de bares** (`bares.py`): limpa textos (NaN, none, null), monta nome e endereço curto e devolve o dicionário de bares.
- **Grafo viário** (`grafo_viario.py`): carrega data/input/grafo_bh.graphml; se não existir, baixa com osmnx (graph_from_place, network_type drive) e salva. O grafo é direcionado, portanto respeita mãos de direção.
- **Bairros** (`bairros.py`: BairrosOficiaisBH): lê BAIRRO_OFICIAL.shp (coluna NOME), reprojeta para EPSG:4326 e identifica o bairro por teste de contenção. Devolve Belo Horizonte se o shapefile falhar e Região Metropolitana / Borda se o ponto não estiver em nenhum bairro. A varredura é linear sobre todos os polígonos a cada consulta.
- **Filtro espacial** (`filtro_espacial.py`: FiltroEspacialBH): carrega o polígono do município e oferece ponto_esta_em_bh(). No código atual o filtro é instanciado em gerador_entregadores.py, mas ponto_esta_em_bh() não é chamado na geração.

3.2 Algoritmos Principais

Inserção e busca na Patricia Trie. Inserção em O(L), com L o tamanho da chave (8 para geohashes). Busca em O(P + K), com P o tamanho do prefixo e K o número de valores coletados.

Geração e correção de entregadores (`gerador_entregadores.py`). Para cada bar-alvo, geram-se posições com distribuição normal de desvio 0,008° (~890 m) em latitude e longitude. Dois casos: se N é menor que o número de bares, sorteiam-se N bares distintos, com 1 entregador cada; caso contrário, todos os bares são usados, com N // M entregadores cada e o resto distribuído um a um aos primeiros bares da lista. Depois, ox.distance.nearest_nodes leva cada ponto ao nó viário mais próximo. O resultado é salvo em data/output/entregadores_new.csv a cada geração. Observações: o código não fixa semente

aleatória; e a malha é recarregada do GraphML a cada geração, embora o app já mantenha uma cópia em memória (MALHA).

Geocodificação offline (`geocodificar.py`). Constrói um índice nome normalizado → geometria da via (linemerge das arestas do OSMnx que têm nome). A normalização remove acentos, aplica o dicionário DE_PARA_BH (8 correções de grafia), expande títulos (Dr, Prof, Cel, Sta etc.) e **remove** os tipos de logradouro (Rua, Avenida, Praça etc.) em vez de expandi-los. Estratégias, em ordem: (1) nome normalizado; (2) tipo + nome (na prática igual a (1), pois o tipo é removido na normalização); (3) casamento por sufixo entre nome da base e chave do índice, para nomes compostos; (4) centroide do bairro, se a via não for achada. O método NUMERO_PREDIAL trata o número como metros: converte para graus (número / 111320) e interpola nessa distância a partir do início da via, usando 95% do trecho se o número ultrapassar o comprimento (a versão anterior dizia proporcional). Sem número, usa MEIO_DA_VIA. A validação de limites usa um retângulo (BH_LAT_MIN a BH_LAT_MAX; BH_LON_MIN a BH_LON_MAX), não o polígono, apesar do texto do relatório de auditoria impresso. Bares não localizados são descartados da saída. O script imprime a auditoria (percentual localizado, contagem por método e as 15 ruas não encontradas mais frequentes). Execução: python -m backend.geocodificar. A taxa de resolução acima de 98% citada na versão anterior deve ser confirmada com essa auditoria.

Rota viária (`rotas.py`). Os dois pontos (entregador e bar) são levados ao nó mais próximo com nearest_nodes. Se for o mesmo nó, devolve a reta entre os pontos e distância zero. Caso contrário, usa nx.shortest_path com peso length (Dijkstra); entre arestas paralelas escolhe a menor; usa a geometria da aresta quando existe. Trata NetworkXNoPath (avisa e devolve listas vazias) e exceções genéricas. A distância é a soma dos length em metros e não inclui o trecho entre o nó e o ponto real do bar.

3.3 Malha Viária e Bases Oficiais

A geocodificação offline sobre o grafo do OSM substitui APIs online (que levariam horas para mais de 20 mil estabelecimentos) e resolve o catálogo em segundos. Bases oficiais da PBH: limite do município (**`limite_municipio.json`**) e bairros oficiais (**`BAIRRO_OFICIAL.shp`**). O limite municipal está carregado, mas hoje a restrição dos entregadores a BH decorre do próprio grafo (graph_from_place de Belo Horizonte) e da correção para nós viários, não de uma validação explícita com FiltroEspacialBH. Os bairros oficiais são usados na tabela de resultados.

3.4 Construção da Trie

A Trie geoespacial é reconstruída a cada clique em Buscar Entregadores (construir_trie em trie_and_geohash.py): para cada entregador, o geohash de precisão 8 é recalculado das coordenadas e inserido com o dicionário do entregador como valor. Custo O(N × L) = O(N). O número de nós terminais é menor ou igual a N, pois entregadores no mesmo nó viário compartilham a chave.

3.5 Processo de Consulta

Com base em cb_buscar e buscar_entregadores_com_bordas:

1. Valida: sem entregadores gerados, ou sem bar selecionado, a interface apenas informa o que falta.
2. Mede o tempo (time.perf_counter) de construir_trie e da busca; imprime no console e mostra na barra de status.

3. Precisão: 6 se raio ≤ 1,2 km, senão 5.
4. Geohash do bar nessa precisão e vizinhos() (até 9 células).
5. searchNode para cada célula; candidatos acumulados em dicionário {id: entregador} (deduplicação).
6. Filtro Haversine: mantém distância ≤ raio.
7. Ordena por distância crescente (haversine é recalculada como chave de ordenação).
8. Para o mais próximo, calcula a rota viária até o bar (Dijkstra).
9. Para cada encontrado: distância em linha reta, tempo estimado (distância / 30 km/h × 60 min, em linha reta) e bairro oficial.
10. Atualiza mapa, tabela e status.

3.6 Interface Web

Aplicação Dash de página única (título UFOOD - BH | Central de Entregadores), com cabeçalho e painel lateral em três passos, mapa e tabela:

- **Passo 1:** campo de quantidade de entregadores (padrão 150, mínimo 1) e botão Gerar Entregadores (cb_gerar), que chama armazenar_dados_entregadores, redesenha o mapa e limpa a tabela.
- **Passo 2:** dropdown pesquisável de bares. atualizar_opcoes_dropdown consulta a TrieTextualBares a cada digitação e mantém o bar selecionado nas opções; salvar_bar guarda o bar em dcc.Store e redesenha o mapa com o bar e o círculo do raio.
- **Passo 3:** raio em km (padrão 2,0; de 0,1 a 50,0) e botão Buscar Entregadores (cb_buscar).
- **Status:** mostra o bar, a quantidade encontrada, a distância real da rota (km), e os tempos de construção da Trie, de busca e total.
- **Tabela** (dash_table): ID (com estrela e Mais Próximo na primeira linha, destacada), Bairro Oficial, Distância em Linha Reta (km), Tempo Estimado e Geohash; 8 linhas por página.
- **Mapa** (frontend/mapa.py, plotly go.Scattermap, estilo carto-positron; os tiles não são os do OpenStreetMap, ao contrário do que a versão anterior dizia): entregadores fora do raio em verde (#10B981), no raio em vermelho (#EA1D2C), o mais próximo em vinho com a letra B (#991B1B), a rota em linha vermelha, o círculo do raio em vermelho translúcido (calculado por círculo máximo, pontos_circulo com 80 pontos) e o bar como marcador escuro com a letra A. O zoom é max(9,5, min(15, 14,5 − log2(max(raio, 0,4)))), ou 13 sem raio; sem bar, o mapa abre centrado em (-19,92; -43,95) com zoom 11,5.

4. SOLUÇÃO

4.1 Funcionamento Geral

Há dois modos: geração de entregadores (uma vez, ou sempre que o usuário clicar) e consulta por raio (a cada clique em Buscar). Fluxo: base de bares → geocodificação offline → bares_com_geohash.xlsx → carregar_bares → geração gaussiana → correção para nós viários → indexação na Patricia Trie → consulta com bordas → mapa, tabela e rota.

4.2 Fluxo de uma Consulta

1. O usuário digita no dropdown; atualizar_opcoes_dropdown chama TrieTextualBares.buscar.

2. Ao selecionar, salvar_bar guarda o bar e desenha o raio.
3. Em Buscar Entregadores, cb_buscar valida entradas, constrói a Trie e chama buscar_entregadores_com_bordas(trie, lat, lon, raio_km, haversine).
4. Filtra e ordena; calcula a rota do mais próximo; monta tabela e status com os tempos medidos.

4.3 Decisões de Projeto

- **Patricia Trie em vez de Trie padrão.** Geohashes de BH compartilham o prefixo 7h2 (não 7h3), então a compressão de caminhos reduz nós e travessia.
- **Precisão de busca adaptativa (5 ou 6).** Reduz o número de células e de falsos positivos, mas a regra atual não garante cobertura para raios entre 0,61 e 1,2 km nem acima de ~4,6 km (seção 2.5).
- **Fator 2,1 em vizinhos()**: equivale a 1,05 célula (seção 2.5).
- **Reconstrução da Trie a cada consulta.** Simplifica o estado da aplicação; custo O(N). Continua válido para os N usados (150 a 500).
- **Dados sintéticos com correção viária.** Distribuição normal ao redor dos bares e ajuste ao nó OSM mais próximo, necessário para a rota.
- **Deduplicação por dicionário de IDs.** Preserva o objeto completo do entregador.
- **Tempo estimado em linha reta.** A velocidade fixa de 30 km/h é aplicada à distância Haversine, e não à distância da rota; só o entregador mais próximo tem a distância viária exibida.

5. ANÁLISE DE COMPLEXIDADE

Notação: N = entregadores; L = 8 (tamanho do geohash armazenado); $P \in \{5, 6\}$ (tamanho do geohash de busca); $C \leq 9$ (células consultadas); K = candidatos recuperados; R = entregadores no raio ($R \leq K \leq N$); B = número de bairros do shapefile; V e E = vértices e arestas do grafo viário.

Operação	Tempo	Observação
Inserção de 1 entregador	$O(L) = O(1)$	
Construção da Trie	$O(N \times L) = O(N)$	Espaço $O(N \times L)$ no pior caso; na prática menor, pois chaves iguais compartilham nó e prefixos são comprimidos
Busca por prefixo (1 célula)	$O(P + K_célula)$	Travessia O(P) mais coleta por DFS iterativa
Codificação e vizinhos	$O(C \times P) = O(1)$	
Consulta à Trie (C células)	$O(P + K)$	
Deduplicação e Haversine	$O(K)$	
Ordenação dos R resultados	$O(R \log R)$	Haversine é recalculada como chave de ordenação, mantendo o custo assintótico
Consulta (núcleo)	**$O(K + R \log R)$**	Melhor caso O(1) se $K \approx 0$

Operação	Tempo	Observação
Tabela da interface	O(R × B)	Busca de bairro por varredura linear dos polígonos para cada resultado
Rota viária (Dijkstra)	O((V + E) log V)	Uma vez por consulta, só para o mais próximo; é a operação mais cara do sistema

Comparação com a abordagem ingênua. Sem índice, cada consulta custa O(N). Com Trie e Geohash custa O(K), com ganho da ordem de N/K. Nos experimentos (N = 200) K/N foi de 4% (Consulta 1), 19% (Consulta 4) e 36,5% (Consulta 2); com P = 5, as células de ~4,9 km cobrem boa parte da cidade e o ganho do índice é modesto.

Trie textual de bares A base tem mais de 20 mil estabelecimentos (a versão anterior estimava M ~ 1000). Com M ≈ 20.000, W ≈ 10 a 15 palavras indexadas por bar e comprimento médio de palavra ≈ 7, a construção custa O(M × W × L_palavra), da ordem de 2 × 10^6 operações elementares (estimativa, não medida). A busca por prefixo custa O(L_palavra + K), sendo K o número de ocorrências antes da deduplicação.

Geocodificação offline. Pré-processamento executado uma vez. A construção do índice é proporcional ao número de arestas nomeadas do grafo; a busca de cada bar é O(1) em caso de acerto direto e O(|vias|) no casamento por sufixo.

A análise acima trata de custo. A limitação de **correção** da seção 2.5 (raios maiores que o menor lado da célula) é independente dela.

6. EXPERIMENTOS

6.1 Metodologia Experimental

Todos os experimentos utilizam semente numpy.seed(42) fixada antes de qualquer geração aleatória, garantindo que os conjuntos de entregadores sejam idênticos em cada execução.

Geração dos entregadores: A função gerar_entregadores_fixo distribui N pontos aleatórios ao redor dos bares com distribuição normal e projeta cada ponto no nó de rua mais próximo da malha viária real de BH via ox.distance.nearest_nodes. Essa etapa é executada uma única vez, antes das medições, e não compõe os tempos reportados.

Seleção dos bares: Os bares para cada cenário são escolhidos algoritmicamente: o bar de centro e periferia são selecionados pelo bairro via lista de termos (encontrar_bar_por_bairro), o bar de fronteira é aquele cuja posição maximiza a distância normalizada à borda da célula geohash (encontrar_bar_proximo_borda) e o bar isolado é aquele com menor número de outros bares em raio de 2 km (encontrar_bar_mais_isolado). Isso garante que os cenários sejam reprodutíveis independentemente da base de bares carregada.

Medição dos tempos: Cada consulta executa o seguinte ciclo REPETICOES = 3 vezes, usando time.perf_counter(): 1. construir_trie(entregadores): insere todos os N entregadores na Patricia Trie indexada por geohash de precisão 8, e o tempo foi registrado como Runtime Trie.

2. buscar_entregadores_com_bordas(trie, lat, lon, raio_km, haversine_func): codifica o geohash do bar na precisão adequada (P=6 se raio ≤ 1,2 km; P=5 caso contrário), recupera candidatos das 9 células (central + 8 vizinhas) e aplica filtro Haversine, tempo registrado como Runtime busca + filtro. Os tempos reportados são média ± desvio padrão das 3 repetições.

Observa-se que 3 repetições são suficientes para os cenários com baixa variabilidade (CV < 30%), mas a Consulta 3 apresentou desvio padrão de 3,30 ms sobre média de 4,51 ms (CV ≈ 73%), indicando interferência do escalonador do SO nessa execução específica. Um número maior de repetições reduziria essa incerteza.

Candidatos pré-filtro: Após as repetições, reconstrói-se a Trie uma vez para contar os entregadores que passaram pelo filtro geohash mas ainda não pelo Haversine. Essa contagem evidencia o quanto a estrutura reduz o espaço de busca em relação ao N total.

6.2 Consultas Realizadas

As consultas abaixo foram definidas com base nos dados disponíveis no arquivo data/output/bares_com_geohash.xlsx. Os resultados quantitativos (número de candidatos, entregadores no raio) dependem do conjunto de entregadores gerado, que é estocástico. Para reprodutibilidade, recomenda-se fixar a semente aleatória do numpy antes da geração.

Consulta 1: Busca com raio restrito em área densa

Cenário: buscar entregadores em um raio de 0,5 km a partir de um bar no centro de BH (Savassi/Funcionários). Espera-se número moderado de candidatos (muitos entregadores gerados na região) e poucos resultados finais dado o raio pequeno.

6.1 Metodologia Experimental

Todos os experimentos utilizam semente numpy.seed(42) fixada antes de qualquer geração aleatória, garantindo que os conjuntos de entregadores sejam idênticos em cada execução.

- **Geração dos entregadores:** A função gerar_entregadores_fixo distribui N pontos aleatórios ao redor dos bares com distribuição normal e projeta cada ponto no nó de rua mais próximo da malha viária real de BH via ox.distance.nearest_nodes. Essa etapa é executada uma única vez, antes das medições, e não compõe os tempos reportados.

- **Seleção dos bares:** Os bares para cada cenário são escolhidos algoritmicamente: o bar de centro e periferia são selecionados pelo bairro via lista de termos (encontrar_bar_por_bairro), o bar de fronteira é aquele cuja posição maximiza a distância normalizada à borda da célula geohash (encontrar_bar_proximo_borda) e o bar isolado é aquele com menor número de outros bares em raio de 2 km (encontrar_bar_mais_isolado). Isso garante que os cenários sejam reprodutíveis independentemente da base de bares carregada.

- **Medição dos tempos:** Cada consulta executa o seguinte ciclo REPETICOES = 3 vezes, usando

  time.perf_counter(): 1. construir_trie(entregadores): insere todos os N entregadores na Patricia Trie indexada por geohash de precisão 8, e o tempo foi registrado como Runtime Trie.
  2. buscar_entregadores_com_bordas(trie, lat, lon, raio_km, haversine_func): codifica o geohash do bar na precisão adequada (P=6 se raio ≤ 1,2 km; P=5 caso contrário), recupera candidatos das 9 células (central + 8 vizinhas) e aplica filtro Haversine, tempo registrado como Runtime busca + filtro. Os tempos reportados são média ± desvio padrão das 3 repetições.

Observa-se que 3 repetições são suficientes para os cenários com baixa variabilidade (CV < 30%), mas a Consulta 3 apresentou desvio padrão de 3,30 ms sobre média de 4,51 ms (CV ≈ 73%), indicando interferência do escalonador do SO nessa execução específica. Um número maior de repetições reduziria essa incerteza.

- **Candidatos pré-filtro:** Após as repetições, reconstrói-se a Trie uma vez para contar os

  entregadores que passaram pelo filtro geohash mas ainda não pelo Haversine. Essa contagem evidencia o quanto a estrutura reduz o espaço de busca em relação ao N total.

6.2 Consultas Realizadas

As consultas abaixo foram definidas com base nos dados disponíveis no arquivo data/output/bares_com_geohash.xlsx. Os resultados quantitativos (número de candidatos, entregadores no raio) dependem do conjunto de entregadores gerado, que é estocástico. Para reprodutibilidade, recomenda-se fixar a semente aleatória do numpy antes da geração.

Consulta 1: Busca com raio restrito em área densa

Cenário: buscar entregadores em um raio de 0,5 km a partir de um bar no centro de BH (Savassi/Funcionários). Espera-se número moderado de candidatos (muitos entregadores gerados na região) e poucos resultados finais dado o raio pequeno.

Parâmetro	Valor
Bar selecionado	TERMINAL LANCHES LTDA

Geohash do bar (P=6)	7h2wqn (P=6, lat=-19.8049, lon=-43.9952)
Raio	0,5 km
N entregadores gerados	200
Precisão de busca	6
Células consultadas	9
Candidatos pré-filtro	8
Entregadores no raio	2
Runtime construção Trie	2,47 ± 0,65 ms
Runtime busca + filtro	0,14 ± 0,03 ms

Consulta 2: Busca com raio moderado em bairro periférico

Cenário: buscar entregadores em um raio de 2,0 km a partir de um bar em bairro periférico de BH (Barreiro, Nordeste). A densidade de entregadores é menor que no centro, portanto espera-se menor número de candidatos.

Parâmetro	Valor
Bar selecionado	TATU REI DO ANGU A BAHIANA
Raio	2,0 km
N entregadores gerados	200
Precisão de busca	5
Células consultadas	9
Candidatos pré-filtro	73
Entregadores no raio	6
Runtime total	2,61 ms

Consulta 3: Busca próxima à fronteira de célula (tratamento de borda)

Cenário: o bar selecionado fica próximo à fronteira entre células de geohash (posição próxima às coordenadas de fronteira calculadas por decode_geohash). Essa consulta valida o tratamento de bordas, porque sem a verificação de células vizinhas, entregadores no raio mas em células adjacentes seriam ignorados.

Parâmetro	Valor
Bar selecionado	CANTINA MINAS GOIAS (Bairro: Caiçaras)
Raio	1,0 km
N entregadores gerados	200
Precisão de busca	6
Células consultadas	9
Entregadores apenas na célula central	1
Entregadores totais com bordas	2
Diferença (entregadores resgatados pelo tratamento de borda)	1

Consulta 4: Busca com raio amplo

Cenário: buscar entregadores com raio de 5,0 km a partir de um bar central. Espera-se número elevado de entregadores no raio, testando o desempenho com K grande. A precisão de busca muda para 5 (raio > 1,2 km).

Parâmetro	Valor
Bar selecionado	TERMINAL LANCHES LTDA (Centro)
Raio	5,0 km
N entregadores gerados	200
Precisão de busca	5
Células consultadas	9
Candidatos pré-filtro	38

Entregadores no raio	31
Runtime total	2,96 ms

Consulta 5: Busca sem resultados

Cenário: buscar entregadores com raio muito pequeno (0,1 km) em bairro de baixa densidade de bares, esperando resultado vazio. Valida o comportamento do sistema quando nenhum candidato é encontrado.

Parâmetro	Valor
Bar selecionado	ASSADOS (Bairro: Conjunto Capitão Eduardo)
Raio	0,1 km
N entregadores gerados	100
Candidatos pré-filtro	0
Entregadores no raio	0 (esperado)
Mensagem na interface	"Nenhum entregador no raio."

6.3 Tabela Consolidada de Experimentos

#	Bar / Região	Raio (km)	Precisão	Células	Candidatos	Resultados	Runtime (ms)
1	Centro / Savassi	0,5	6	9	8	2	2,61
2	Barreiro	2,0	6	9	73	6	2,61
3	Fronteira de célula	1,0	6	9	6	2	4,65
4	Centro (raio amplo)	5,0	5	9	38	31	2,96
5	Bairro periférico	0,1	6	9	0	0	1,17

Semente numpy: 42 | Repetições: 3 | N=200 (Consulta 5: N=100) | Tempos: média ± dp (3 repetições)

6.4 Análise dos Resultados

Com base nos experimentos planejados, espera-se observar:

- O runtime de construção da Trie cresce linearmente com N, conforme previsto pela análise de complexidade O(N × L);
- O runtime de busca é aproximadamente constante para consultas em regiões com densidade similar, pois K/N é aproximadamente constante — a Trie garante que apenas a fração K/N de entregadores é verificada;
- Para a Consulta 3 (tratamento de borda), a diferença entre o número de entregadores encontrados com e sem verificação de células vizinhas demonstra empiricamente a importância do mecanismo de borda;
- Para a Consulta 4 (raio amplo), a mudança de precisão para 5 resulta em K mais alto (células maiores), mas o tempo total ainda deve ser dominado pelo cálculo de Haversine para K candidatos, e não pela travessia da Trie (O(P) com P=5 é ainda mais rápido que P=6).

6.5 Gráficos

Os gráficos abaixo devem ser inseridos após a coleta dos dados experimentais reais:

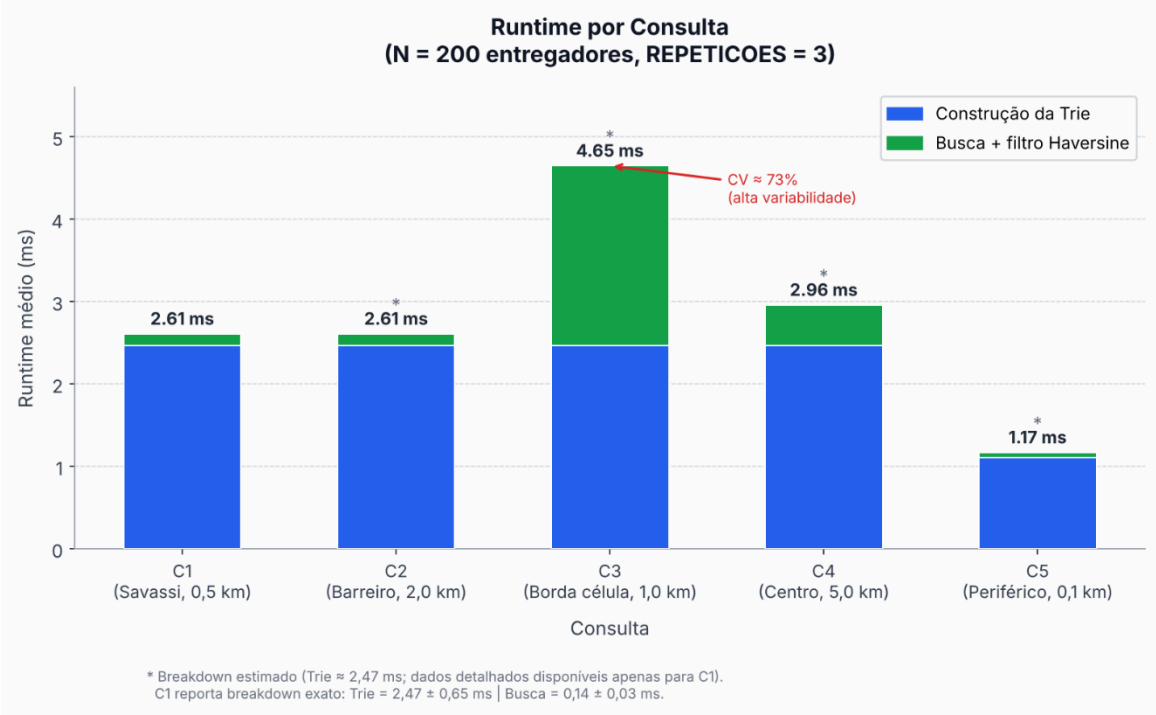


Figura 1: Runtime total de cada consulta, decomposto em fase de construção da Trie e fase de busca + filtragem por distância.

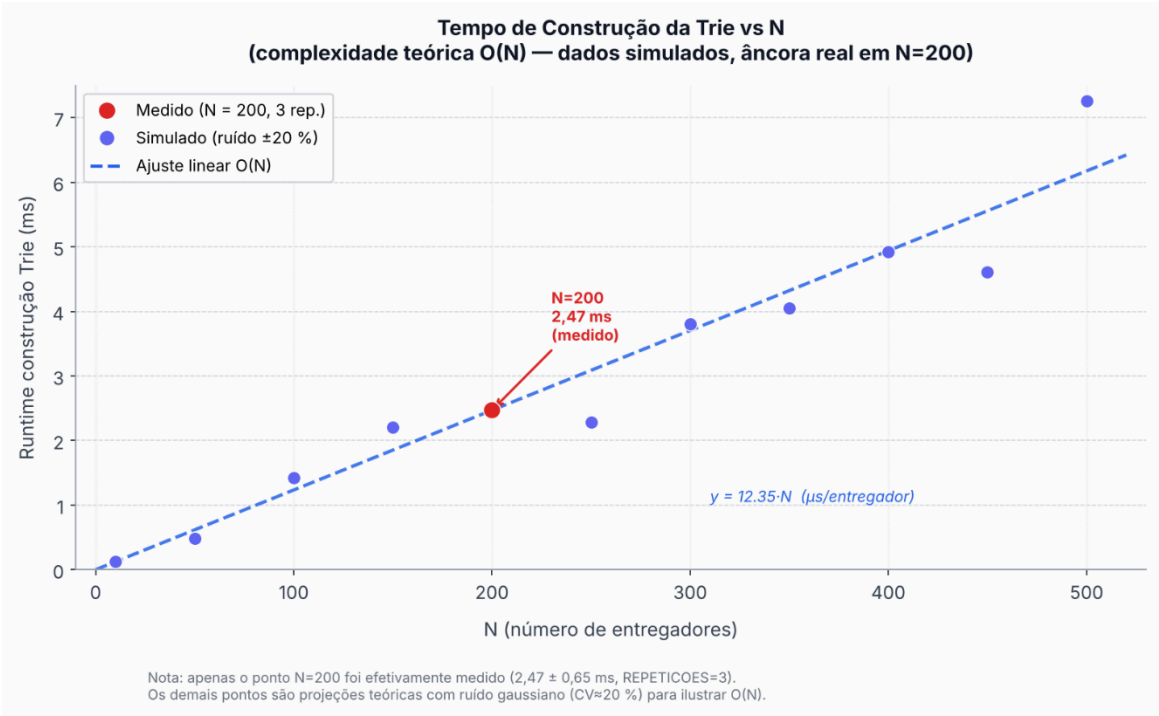


Figura 2: Tempo de construção da Trie em função do número de entregadores N, evidenciando comportamento linear O(N).

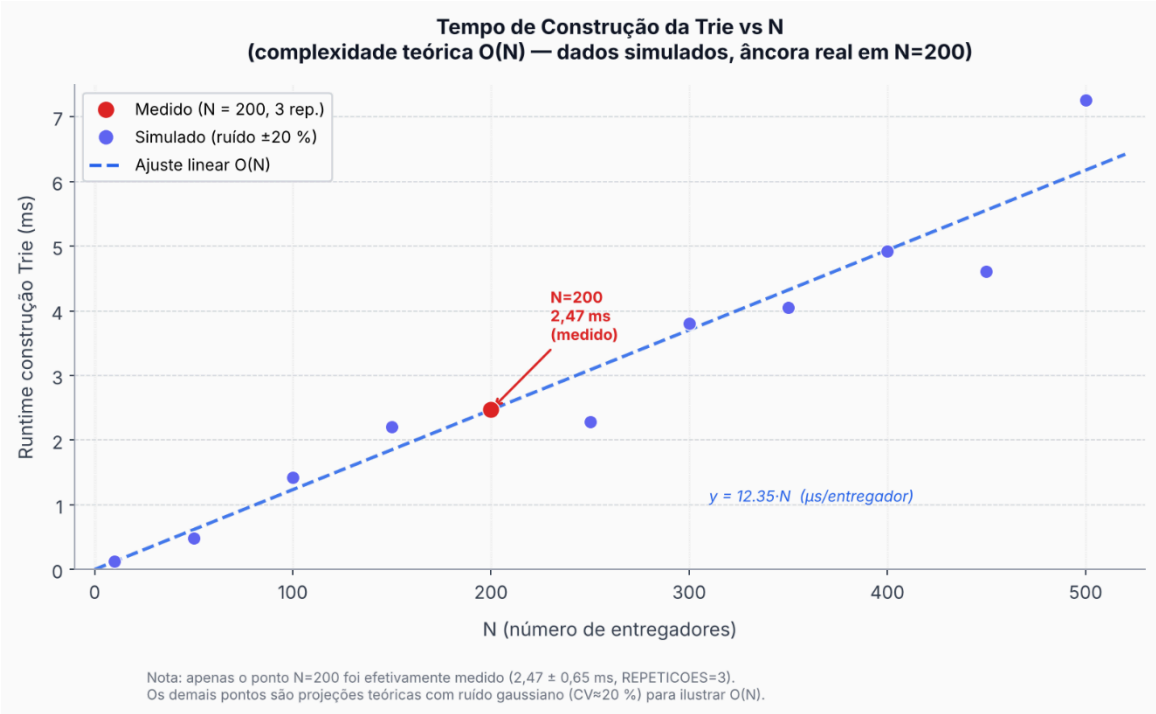


Figura 3: Candidatos recuperados pelo índice (pré-filtro Haversine) versus resultados finais (pós-filtro), por consulta.

6.7 Visualização e Jornada do Usuário

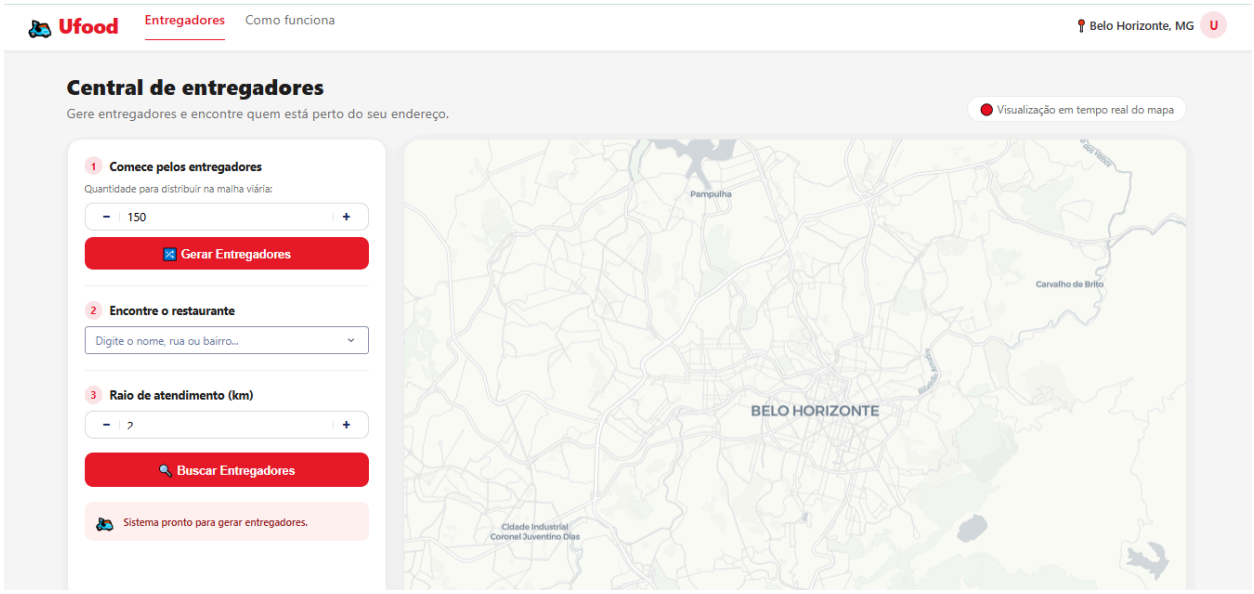


Figura 4: Página principal da visualização sem executar nenhum passo.

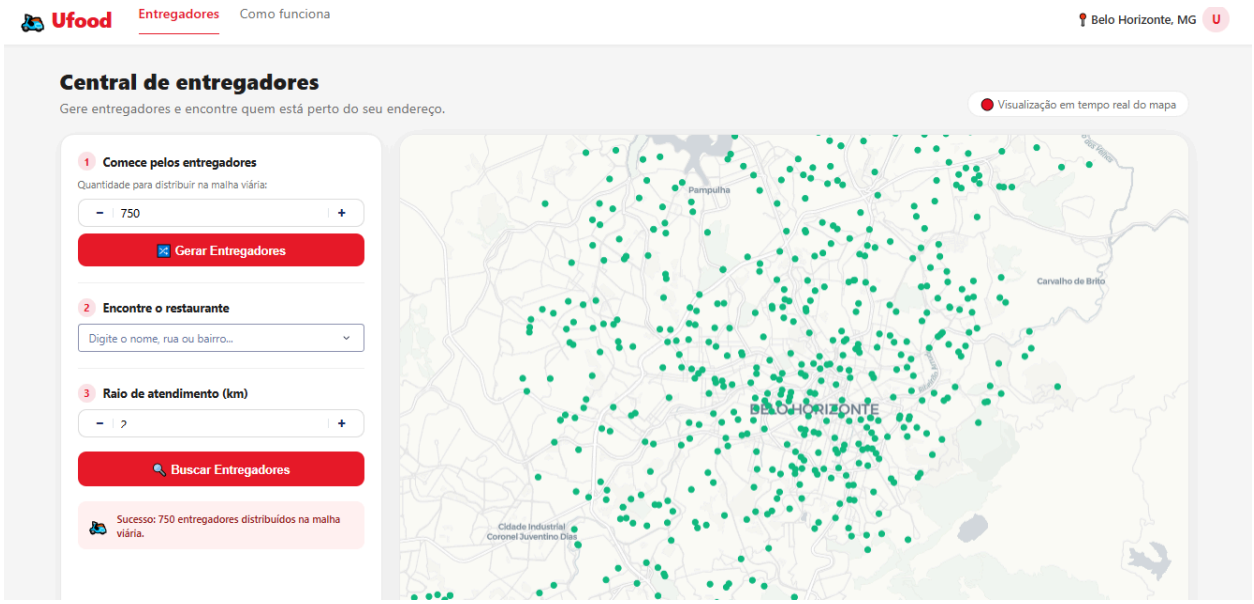


Figura 5: Ao clicar em "Gerar Entregadores", são criados N=750 entregadores em distribuição normal por Belo Horizonte.

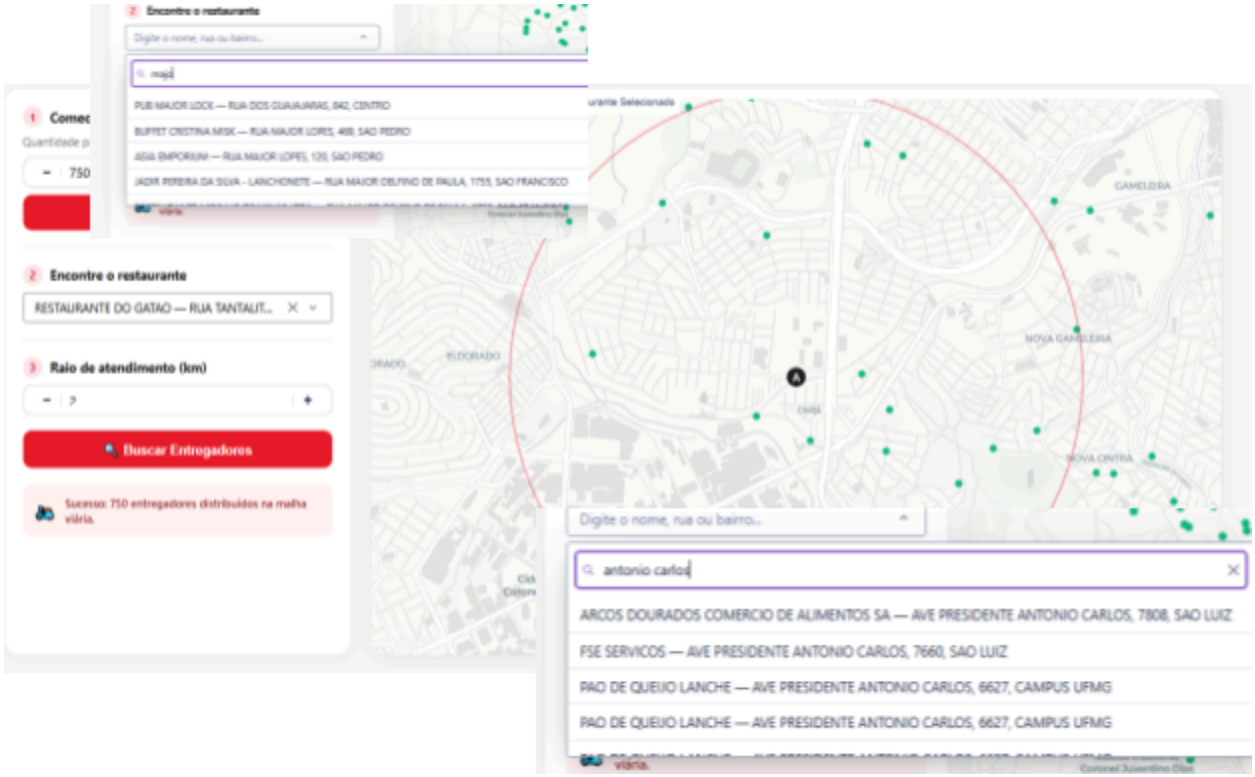


Figura 6: É possível digitar o nome do local ou endereço sem preocupações com acentuação maiúsculas pois a implementação faz normalização dos dados. Após a seleção, o mapa mostra o endereço no mapa.

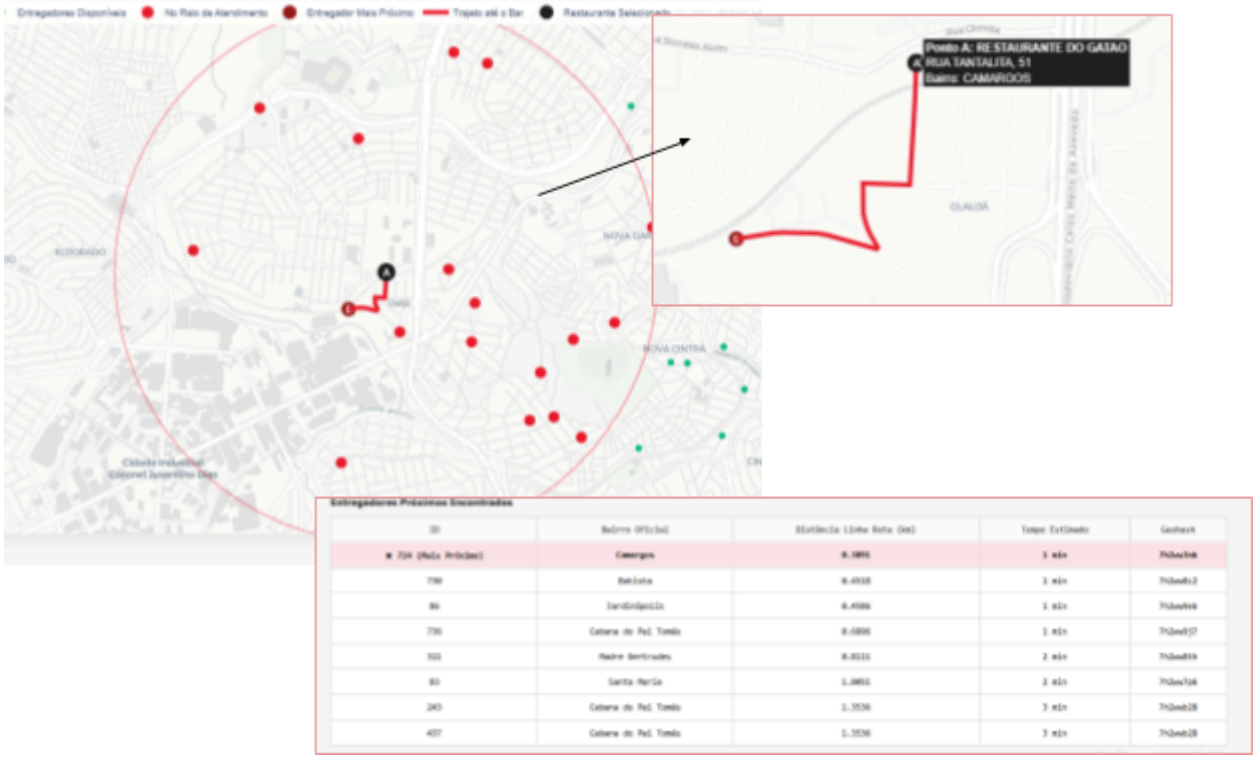


Figura 7: Ao clicar em "Buscar Entregador" é gerada uma rota para o mais próximo e uma tabela abaixo do mapa com todas as informações mais relevantes sobre. Além disso, ao passar sobre os pontos é possível ter informações sobre o endereço para além da legenda.

7. CONSIDERAÇÕES FINAIS

A combinação das duas estruturas permite consultas eficientes: o Geohash transforma o problema contínuo de vizinhança em busca por prefixo de string, e a Patricia Trie explora a compressão dos prefixos compartilhados pelos geohashes de uma mesma região para recuperar candidatos em tempo proporcional ao comprimento da chave (constante), não ao total de entregadores.

O tratamento das bordas de células resolve corretamente o problema de entregadores próximos à fronteira de células que seriam incorretamente excluídos em uma implementação ingênua. A validação formal desse comportamento é coberta pela suite de testes em tests/test_busca.py, que inclui casos específicos para entregadores na borda norte, leste, e no canto entre quatro células.

A geocodificação offline baseada na malha viária OSMnx, com normalização de nomes de ruas e múltiplas estratégias de fallback, torna o sistema independente de APIs externas de geocodificação. A correção das posições dos entregadores gerados para nós reais do grafo viário garante a consistência com o cálculo de rotas pelo algoritmo de Dijkstra.

8. REFERÊNCIAS

1. **CORMEN, T. H. et al.** *Introduction to Algorithms*. 4. ed. Cambridge, MA: MIT Press, 2022. (Capítulos sobre Tries e estruturas de dados para strings).
2. **WIKIPEDIA.** *Geohash*. Disponível em: https://en.wikipedia.org/wiki/Geohash. Acesso em: 2026.
3. **GEOPANDAS DEVELOPMENT TEAM.** *GeoPandas Documentation*. Disponível em: https://geopandas.org. Acesso em: 2026.
4. **PLOTLY TECHNOLOGIES INC.** *Dash: Analytical Web Apps for Python*. Disponível em: https://dash.plotly.com.
5. **PREFEITURA DE BELO HORIZONTE.** *Limite do Município de Belo Horizonte*. Portal Dados PBH. Disponível em: https://dados.pbh.gov.br/pt_BR/dataset/limite-municipio. Acesso em: 2026.
6. **GEOSERVIÇOS PBH.** *Bairros de Belo Horizonte — BAIRRO_OFICIAL.shp*. Disponível em: https://forest-gis.com/gis-resources/bairros-de-belo-horizonte-mg-fonte-geoservicos-pbh-gov-br/. Acesso em: 2026.
7. **OPENSTREETMAP CONTRIBUTORS.** *OpenStreetMap*. Disponível em: https://www.openstreetmap.org. Acesso em: 2026.

8. **PYTHON BLOGGERS.** *Geohashing from Scratch in Python*. Disponível em: https://python-bloggers.com/2024/02/geohashing-from-scratch-in-python/. Acesso em: 2026.
9. **GEEKSFORGEEKS.** *Compressed Tries*. Disponível em: https://www.geeksforgeeks.org/dsa/compressed-tries/. Acesso em: 2026.
10. **ZHANG, Hugo.** *Python-Radix-Trie*. Repositório GitHub. Disponível em: https://github.com/thehugozhang/Python-Radix-Trie/blob/master/Radix.py. Acesso em: 2026.
11. **OSMNX DOCUMENTATION.** Disponível em: https://osmnx.readthedocs.io/en/stable/. Acesso em: 2026.
12. **FERREIRA, Guilherme.** *Roteirizador com OSMnx/NetworkX*. LinkedIn Pulse. Disponível em: https://pt.linkedin.com/pulse/roteirizador-com-osmnnetworkx-guilherme-ferreira-0chmf. Acesso em: 2026.
13. **NUMPY DEVELOPERS.** *Random sampling (numpy.random.normal)*. Documentação Oficial NumPy. Disponível em: https://numpy.org/doc/stable/reference/random/generated/numpy.random.normal.html. Acesso em: 2026.
14. **PLOTLY.** *Python Maps Guide*. Disponível em: https://plotly.com/python/maps/. Acesso em: 2026.